Survey & Analysis Report

From Prompt to Pose: Semantic Editing of Motion with T2M-GPT[28] and MoMask[6]

Jack Smith
Supervisor: Zhixiang Chen
Module code: COM3610

5 December 2025

Abstract

Recent advances in text-to-motion models have enabled the generation of lifelike human movement from natural language, but these systems lack meaningful editability. This project presents a hybrid motion editing pipeline that combines the language-driven generative capabilities of T2M-GPT[28] with the bidirectional regeneration power of MoMask[6]. Integrated directly into Unity, the tool allows users to generate motion from text prompts, edit selected keyframes, and regenerate surrounding segments with semantic and physical stability. A physics-aware correction layer ensures the plausibility of regenerated motion. The system is designed for iterative workflows used by animators and game developers, bridging the gap between generative AI and practical creative tools. A stretch goal explores a novel bidirectional masked transformer architecture capable of text-prompted segment regeneration, offering a future direction for editable, language-guided animation systems.

Contents

1 Introduction: Context and Motivation **3**

2 Background and Literature Review **3**

2.1 Text-to-Motion Generation Models . . . 3

2.1.1 Discrete Token-Based Models . . . 3

2.1.2 Masked Generative Models . . . 4

2.1.3 Diffusion-Based Models . . . 4

2.1.4 Large Multimodal Motion Models . . . 5

2.2 Motion Editing and Keyframing Research . . . 5

2.2.1 Keyframe and Sparse Control Techniques . . . 5

2.2.2 Object-Aware Motion Models . . . 6

2.2.3 Industry Tools . . . 7

2.3 Physics-Based and Inverse Kinematic Systems . . . 8

2.3.1 Differentiable Physics Models . . . 8

2.3.2 Inverse Kinematics Approaches . . . 8

2.3.3 Physics-Guided Motion Correction . . . 9
2.4 Datasets for Text–Motion Research . . . 9
2.5 Evaluation Metrics . . . 10
2.5.1 Quantitative Measures . . . 10
2.5.2 Qualitative and User Studies . . . 11

3 Problem Analysis **11**
3.1 Limitations in Controllability . . . 11
3.2 Limitations in Physical Consistency . . . 12
3.3 Limitations in Editing Tools . . . 12
3.4 Summary and Project Direction . . . 13

4 Proposed Solution and Technical Approach **13**
4.1 System Overview . . . 13
4.2 Intended Editing Workflow . . . 13
4.3 Motion Regeneration Module . . . 14
4.4 Physics and Inverse Kinematics Correction Layer . . . 15
4.5 Tools, Libraries and Implementation . . . 16
4.6 Stretch Goal: Text-Prompted Segment Regeneration . . . 16

5 Progress and Evaluation Plan **16**
5.1 Achievements to Date . . . 16
5.2 Evaluation Plan . . . 17
5.2.1 Quantitative Evaluation . . . 17
5.2.2 Embedded Qualitative Evaluation . . . 17
5.3 Timeline and Remaining Tasks . . . 19
5.4 Risks and Mitigation . . . 21

6 Conclusion **21**

1 Introduction: Context and Motivation

Rapid developments in generative modelling have made it possible to produce complex human motion directly from natural language prompts. These systems are being used in animation, virtual production, game development and robotics. In these settings, practitioners require fine-grained control, including the ability to modify parts of an existing motion, retain continuity with surrounding frames, and respect the constraints imposed by bodies and objects.

This report analyses frontier technologies in human motion generation. Attention will be given to methods that support localised motion editing, use of surrounding context during generation, and the reconstruction of short sections within an existing animation. Particular focus will be on text-guided partial regeneration, the ability to regenerate only a segment of an animation based on a text prompt, while preserving continuity with unmodified sections. Maintaining coherence at these boundaries and preserving plausible behaviour remain central challenges.

The review builds the foundation for a system that integrates research-grade models into a system which provides animators, game designers and engineers with the tools to use and edit what they produce. The core aim is to allow a user to generate an animation within the Unity game development engine, and fine-tune it using keyframes, allowing a user to refine specific regions of an animation while keeping the continuity of the motion intact. This editing layer operates on motions already produced by a text-to-motion model. A further extension of the project, treated as a stretch goal, is to develop a context aware model capable of regenerating a selected segment using both a text prompt and the neighbouring frames and context. This extension offers prompt based regeneration of internal segments which is not supported by current models.

2 Background and Literature Review

Research on producing human motion from text prompts has developed into several fundamental branches, each offering benefits and trade-offs in motion representation, controllability, and generative quality. Most systems follow a shared structure in which a language encoder maps written descriptions to semantic features, before a motion generator produces pose trajectories conditioned on those features. Model families differ in the ways motion is represented, how temporal structure is modelled, and how constraints are enforced during generation.

2.1 Text-to-Motion Generation Models

2.1.1 Discrete Token-Based Models

Discrete, token-based motion has become extremely popular due to its compatibility with transformer architectures, which mirror the structure of language models. T2M-GPT [28] follows this approach by using a vector-quantised autoencoder (VQ-VAE) to convert motion

clips into sequences of tokens. The encoder maps each motion frame to a continuous latent vector, which is matched to the nearest entry in a learned codebook. Each entry is assigned an index, so the motion sequence becomes a series of discrete IDs. The decoder reconstructs the motion from these IDs, and the codebook is refined during training.

Once motion is expressed as tokens, a transformer predicts new sequences conditioned on text. It uses self-attention to model relationships across the entire sequence, aligning with semantics extracted from a CLIP-based text encoder. This alignment ensures the generated motion matches the description.

This tokenised format also supports editing. A region can be masked and reconstructed using a model trained for infilling. However, T2M-GPT[28] is autoregressive—it generates from left to right, using only past context. This limits its ability to regenerate middle segments where future frames are known.

In contrast, span-masked models can access both past and future context, learning to infill missing intervals that smoothly connect to surrounding motion. This is ideal for editing tools that replace specific spans while keeping the rest of the motion intact.

2.1.2 Masked Generative Models

Masked generative models reconstruct missing parts of a sequence from known context. MoMask [6] follows this approach by masking temporal spans and joints, training a bidirectional transformer to restore the full sequence. During training, selected segments are replaced with mask tokens, and the surrounding frames guide the prediction.

MoMask[6] encodes motion as latent vectors that capture joint rotations, global movement, and dynamics. These are combined with mask indicators that specify which parts to regenerate. The model uses bidirectional self-attention to ensure masked tokens align temporally with visible context.

This setup supports keyframe editing: edited frames are inserted, and nearby spans are masked. MoMask[6] regenerates the surrounding motion to ensure smooth transitions into and out of the modified pose.

However, MoMask[6] lacks a language encoder and cannot generate motion from text prompts. It is purely a reconstruction model. In cases where users want to replace an interval with a new action (e.g., a roll), MoMask[6] is not sufficient. For these cases, a new or adapted text-to-motion model is needed. For keyframe-based segment regeneration, though, MoMask[6] is highly effective.

2.1.3 Diffusion-Based Models

Diffusion models generate motion by gradually denoising an initial noisy sequence. Models like MDM [21] and MotionDiffuse [30] follow this process, with CLIP-based text embeddings guiding each step to match the prompt.

These models produce high-quality, smooth motion. The iterative denoising process encourages long-term consistency and reduces jitter. However, sampling is slow—many steps are needed to denoise a sequence. Methods like DDIM (Denoising Diffusion Implicit Models) help, but transformer-based models are typically faster.

Diffusion supports soft constraints, like fixing poses or guiding trajectories. But enforcing hard constraints, such as regenerating a fixed segment between two known points, is complex. Additional control networks or anchoring strategies are needed, increasing the computational burden. This makes diffusion excellent for offline synthesis but less ideal for real-time editing.

2.1.4 Large Multimodal Motion Models

Recent models integrate motion, text, and sometimes video into shared spaces. MotionCLIP [22] uses CLIP-like training to align motion with text and video, supporting retrieval and conditioning tasks. MotionGPT [11] expands this with tokenised motion and unified transformers for generation, captioning, and translation.

OmniMotion [14] extends this by grounding motion in video, improving semantic alignment. These systems are powerful for generation and retrieval but do not offer tools for fine-grained editing or seamless reconstruction. Their focus is on high-level semantics rather than frame-level continuity, so they are not suited for segment editing workflows.

2.2 Motion Editing and Keyframing Research

2.2.1 Keyframe and Sparse Control Techniques

Keyframe based motion editing **centres** around manipulating a small set of poses that act as anchors for an animation sequence. Early approaches relied on geometric or kinematic interpolation, where the system computes intermediate frames that link specified poses. These methods allow animators to adjust individual frames, however they require additional smoothing or manual correction to avoid artefacts such as foot sliding or unnatural joint behaviour, and as a result they often require additional correction from an animator [18]. Many early systems used linear blend methods or spline based interpolation, but these approaches did not capture the full complexity of human motion.

In recent research, learning based methods have been explored, which treat keyframes as constraints for a generative model. In these systems, the user specifies a few poses as keyframes and the model reconstructs the intermediate motion. Examples include PoseGPT [15] and other sparse control frameworks [9, 17], which learn from large motion datasets and generate transitions that respect the positional layout of the keyframes. This reduces the burden on the user, since only a few key poses need to be specified rather than a full set of animation frames.

Keyframe control has been extended further by incorporating timed constraints, partial joint targets and trajectory information. Some methods include explicit control over the timing of transitions [7], or accept a desired path for the root joint and generate motion that follows

it [27]. Others allow only a subset of joints to be defined, leaving the remaining joints to be inferred by the model. These ideas support flexible editing workflows, since high level adjustments can be translated into consistent motion without requiring the user to manipulate joint rotations individually.

These approaches improve controllability, by allowing modifications to selected poses or regions without regenerating the entire sequence. They focus on pose based interpolation rather than semantic generation from text, and therefore cannot replace part of a sequence on the basis of an instruction such as "perform a forward roll at this point", nor edit a sequence already prompted to do an action. These methods do not guarantee semantic preservation when a user adjusts spatial keyframes. This highlights the need for stabilisation strategies, which will be covered later in the report. The system proposed here aims to combine the strengths of keyframe based control with a text conditioned model capable of generating new content within selected intervals.

2.2.2 Object-Aware Motion Models

Object-aware motion models extend traditional motion models by incorporating information about nearby surfaces, scene geometry or interactable objects. These models create motion that respects contact patterns and environmental constraints, as well as context about the scene. Older approaches used analytical constraints to keep limbs close to designated surfaces or to enforce foot and hand contacts, but these methods needed a lot of manual tweaking and tuning. They could also break down when the environment changed unpredictably [25].

More recent work makes use of methods grounded in machine learning, to generate realistic movement while interacting with the surroundings. Some models incorporate representations of objects directly into the input, such as occupancy grids, signed distance fields or point clouds, allowing the network to learn which parts of a motion should adapt when an object is placed nearby [16]. Others focus on predicting contact events, adjusting joint configurations so that the model's hands, feet and knees align with the target surfaces without penetration or detachment which affects realism [19]. These systems develop understandings of common interactions, for example: how a character would stabilise itself when leaning on a wall or stepping over an obstacle.

Related research goes about creating scene-aware latent spaces where features that describe both motion and environment are encoded in parallel. MotionNet and Interaction Primitives [10, 24] make embeddings that capture relationships between the pose and the surrounding geometry. In doing this, the model can adjust its predictions according to the position of objects in the scene, ensuring that poses remain feasible. These approaches work well in environments where a user can track the position of objects, as well as when the animation can adjust to these changes while retaining natural behaviour.

Object-aware techniques have direct relevance to real editing workflows because they provide the tools for motion to interact and align with the environment without manual adjustment.

When a keyframe is adjusted, the model can determine new contact points or correct motion such that limb movements are plausible. However, these models typically focus on object context rather than semantic generation, and they lack the ability to introduce new actions from text instructions. These models play a part in this project by supporting the editing pipeline, specifically in the physics correction and contact stabilisation stages, compared to being integrated into the generative core of the system.

2.2.3 Industry Tools

Currently, there are few industry tools that provide a motion editing tool, and the ones that are available do not provide as much editing or fine-tuning capabilities as my tool is attempting to provide. Unity Muse Animate [23] is Unity's new AI-driven motion generation and editing tool, which provides text-to-motion generation within the editor. It supports pose refinement, keyframe insertion, and timeline integration, being designed for artists and technical animators to enable rapid prototyping of character motion.

Muse is a powerful tool that allows animators to use state-of-the-art text-to-motion tools, however it lacks fine-grained control beyond basic pose editing and does not have physics-based correction (beyond automated smoothing). This means that generated motion could be unsuitable for a given scene and will require heavy, manual correction. It edits at the animation curve level, not the generative model level, meaning semantic relations in movements could be lost or distorted with heavy editing. Finally, it does not support the regeneration of internal motion segments, with no architecture for aligning with surrounding context frames using bidirectional conditioning.

This also applies to academic settings. Researchers working on text-to-motion models require mechanisms that preserve global coherence when local edits are made. Muse Animate interpolates between keyframes but it cannot account for deeper motion semantics, such as motion, gait patterns, or contact timing. This highlights the fact that there is a gap between current industrial tooling and state-of-the-art research. Segment-aware regeneration is still an open challenge to be solved.

This project aims to fill in the gaps left in current industry tools. Instead of editing animation curves after generation, this project offers a model-aware pipeline where the specified keyframes guide the reconstruction of the surrounding motion through regeneration. The integration of physics correction and inverse kinematics maintains stable contacts, natural joint behaviour and plausible transitions when poses are altered. This approach supports deeper structural editing than Muse Animate, which cannot rebuild motion segments once the user modifies the spatial constraints. The core contribution lies in enabling keyframe-guided regeneration that aligns local edits to a globally consistent sequence, while preserving the semantic integrity of the original action. The stretch goal of selected segment regeneration based on a prompt input pushes the boundaries of current industry standards, greatly enhancing the workflow of those who would use these industry tools.

2.3 Physics-Based and Inverse Kinematic Systems

2.3.1 Differentiable Physics Models

Physics realism is a constant challenge for motion generation models, since models are only trained on motion capture data which don't explicitly enforce balance, contact stability or realistic momentum. This has effects such as foot sliding, unstable contacts, or a drifting root. Differentiable physics solves this problem by embedding a physics simulator inside a neural network, such that physical constraints can be optimised during training.

There are several systems that adopt the strategy of differentiable physics. DReCon [2] uses differentiable dynamics to refine motion dynamics by penalising balance or contact violations. DiffPhy [29] integrates a body model within the denoising step of the diffusion process, this guides samples towards physically plausible trajectories. PhysDiffusion [26] is a more recent approach which combines diffusion priors with physics losses to reduce unrealistic transitions. All of these models integrate physics during training or sampling, not as a post-generation editor tool.

My tool does not need full differentiable simulation, it's a very heavy and expensive process, requiring full dynamic modelling, which quickly becomes entirely impractical for a real-time editing tool. Instead, this project is planned to integrate Inverse Kinematics (IK) and lightweight physics constraints after regenerating transitions around edited keyframes, so the corrective steps are applied to motion that already reflects the model's understanding of the action. By placing IK in the generative loop rather than on top of it, the system preserves semantic intent and avoids distortions associated with curve-level edits. My tool draws inspiration from differentiable physics, but not the full methodology.

2.3.2 Inverse Kinematics Approaches

Inverse Kinematics is an old and well tested technique, centring around the idea that given a desired end-effector position, such as a hand or a foot, solve the intermediate joint angles within some constraints. Forward Kinematics, on the other hand, computes the end-effector position from known joint angles, inverse kinematics solves the reverse mapping, often where there could be multiple joint angle states which produce the same result. This becomes essential in animation editing, where users adjust poses by moving effectors rather than adjusting joint angles. IK is a great tool for adjusting poses, ensuring they stay feasible and consistent, within the context and constraints of the character.

There are a wide range of IK algorithms developed to efficiently solve joint configurations, including Cyclic Coordinate Descent (CCD) [12], FABRIK [1], and Jacobian methods [3]. All of these approaches are commonly found in animation tools and game engines, including Unity's Animation Rigging system, which incorporates IK solvers directly. Their availability and fast performance make them suitable for interactive editing tools that require stable poses while making adjustments.

Inverse kinematics plays an essential role in the core editing workflow developed in this

project. Both after initial generation, and the editing of keyframes. In which immediate surrounding motion is regeneration, IK is applied to enforce feasible joint angles, maintain contact stability and correct deviations introduced.

2.3.3 Physics-Guided Motion Correction

Even with Inverse Kinematics (IK), motion can produce unnatural artefacts. Commonly, these include foot sliding, hyperextension, joint jitter, and incorrect ground contacts. These can occur during generation but are exacerbated during keyframe editing, where the motion priors encoded by the generative model are disrupted. Fully resolving these artefacts would require a full physics simulation, which is computationally expensive and unsuitable for an interactive environment. On the other hand, lightweight physics-guided correction rules are a practical way to restore plausibility. By correcting the largest deviations without relying on a full dynamic solver, these methods achieve stability whilst maintaining performance within the editing loop, making them suitable for use within animation tools.

Foot locking [8] is a powerful way to enhance realism, as it detects contact frames by using velocity thresholds, and ensures foot joints remain fixed relative to the ground to prevent sliding. Root trajectory smoothing [13] is a technique applied to smooth abrupt changes caused by keyframe adjustments, replacing sharp movements with a consistent curve that better represents whole-body momentum. Momentum-consistency approximations smooth velocity changes so that there are no sudden accelerations. Trajectory projection techniques [4] tweak the root path so that it aligns with the edited poses. These methods are computationally far less expensive compared to differentiable physics models, whilst being widely used in game engines and animation tools.

In this project, motion is regenerated using edited keyframes. This regeneration preserves the semantics but physical plausibility may suffer, so applying these physics rules restores grounding and smooth transitions. These techniques can also support segment regeneration as transitions before, after, and during the edited interval must remain natural and feasible.

The proposed pipeline is as follows: the generative model first preserves semantics, the IK maintains natural joint angles, then the physics rules preserve physical feasibility. This combination works in real-time, enabling a fast and responsive editing pipeline, whilst still avoiding the cost of full simulation.

2.4 Datasets for Text–Motion Research

All text-to-motion models require datasets in which motion clips are paired with natural language descriptions. HumanML3D [5] is a common choice. It contains over 14,000 motion sequences, each with a short text description of the actions, poses, and interactions with the environment.

AMASS is a much larger dataset and merges multiple motion-capture datasets. Motion is standardised using the SMPL body model. The main drawback is that it doesn't include

language annotations. It can, however, be useful for pretraining or improving representation learning because it covers many different motion types.

This project needs text-conditioned generation, therefore it needs a dataset with language descriptions. HumanML3D[5] fits because every motion sequence has text paired with it. Its format of positions, rotations, and velocities in a consistent structure pairs well with a transformer pipeline. The dataset covers a large range of everyday actions, which are very relevant to animation and games. AMASS can still be used as a supplementary dataset, as it will provide broader pose distributions, help with general motion modelling and could be useful in testing how keyframe constraints behave.

2.5 Evaluation Metrics

2.5.1 Quantitative Measures

In the evaluation of motion models, you must be able to check objective accuracy. Editing models, like the one proposed here, require metrics that measure continuity across boundaries, realism of the regenerated segments, as well as the alignment to text prompts. No single metric can capture everything, therefore we must use multiple different metrics to evaluate every angle of quality. These quantitative metrics are also essential for comparing alignment and performance against existing models.

There are three core motion quality metrics that are commonly used. The first is Frechet Gesture Distance (FGD), inspired by the Frechet Inception Distance (FID) used in image generation. It measures the distance between generated motion and real motion feature distributions. It uses a trained motion encoder, like the HumanML3D[5] feature extractor, and is used in notable works such as Text-to-Motion GPT (T2M-GPT[28]), the Human Motion Diffusion Model (MDM[20]), and MoMask[6]. Diversity metrics are also used, measuring the variation across multiple samples generated on the same text prompt. This ensures the model doesn't collapse into repeated motions. Finally, there is Average Displacement Error (ADE) and Final Displacement Error (FDE), which commonly evaluate the trajectory or root motion correctness. These are particularly applicable for segment regeneration, where the regenerated segment must correctly join with its neighbours.

There is also the need to evaluate how well the output matches a given description. Text-motion alignment metrics assess whether a generated or regenerated segment corresponds to the semantics of the prompt. Retrieval Consistency Score (RCS) is one of the standard measures used for this, as it can measure how consistently text and motion embeddings match each other. Other alignment metrics used in recent work include top-k retrieval accuracy and CLIP-based similarity scores, which measure how reliably a motion sequence is matched to its corresponding text among alternatives. These metrics help assess semantic correctness that can't be detected through trajectory or pose statistics alone, complementing physical quality metrics very well.

2.5.2 Qualitative and User Studies

Metrics such as realism, believability, style and usability require human judgement. This is especially true in the context of editing workflows, where smoothness of transitions, user control, alignment with creative intent and perceived continuity are all key factors that can't be quantitatively evaluated.

For optimum coverage, the system should be evaluated by both experts and general users. Experts include animators or game designers, who can judge usability, quality of the results and how likely they would be to integrate this tool into their workflow. A general user study could compare between versions, generating and ranking a wide range of different scenarios and motions, while ranking the generation on different qualitative metrics. This is used in works such as MoMask[6], MDM[20] and T2M-GPT[28]. It can also be judged on anecdotal or observational insights, such as how frequently the user needs to correct the model output and how stable the model behaves after repeated edits.

For this tool, qualitative evaluation is essential for providing interactive editing. I will measure whether transitions after keyframe edits feel natural, whether physics corrections reduce artefacts, and whether users feel in control of the regenerated segment. I will employ a small group to test a version of my tool, which allows them to generate then evaluate motions. I will also collect comparisons to Muse Animate and whether this tool is an improvement on the current industry standard. These evaluations are complementary to the quantitative tests, demonstrating the editing workflow is usable by real artists and designers.

3 Problem Analysis

Human motion generation systems have advanced a lot in recent years, yet there remain limitations when used in practical applications. The gap between current capabilities and the requirements in animation tools become clear when examining controllability, physical consistency, as well as the lack of a mechanism for local editing and iteration. These challenges became the motivation for the system developed in this project.

3.1 Limitations in Controllability

Most text-to-motion models operate with a single pass, producing motion based on a prompt, with little capability to edit or modify the result without manual adjustments. T2M-GPT[28] is autoregressive, meaning it predicts the next token using only the tokens that precede it, as well as the text-prompt. This design restricts the ability to regenerate internal segments of a motion as the model cannot use future tokens as conditioning information. The absence of bidirectional context means there is no assurance the regenerated section reconnects naturally with both its past and future frames.

In real editing workflows, there is often the need to adjust a small set of poses rather than the entire motion. Keyframes address this limitation partially by adjusting selected poses.

However, they typically focus on pose interpolation rather than semantic regeneration with respect to a text-prompt. When a motion that is prompted to do a specific action is edited, the semantic meaning encoded in the motion can become distorted or can break. This project introduces keyframe edits that trigger a short-range regeneration process built on top of an autoregressive model, while a fully bidirectional architecture is recognised as a more ambitious but more accurate extension explored later in the report. These methods should maintain semantic integrity by regenerating only the affected region while keeping the remainder of the motion unchanged.

3.2 Limitations in Physical Consistency

Even if the generation process produces plausible motion, subsequent editing can break physics. There are many variables in which the motion can violate physics, such as joint instability, ground penetration, incorrect contact handling and abrupt changes in velocity. Industry tools such as Muse Animate apply smoothing and curve-level IK after generation, however these corrections do not account for the semantics of the motion or the physical assumptions embedded in the original model.

Moreover, most text-to-motion systems are not trained on physical constraints but statistical regularities, which can easily break in editing. Diffusion models can sometimes mitigate this by producing globally smooth motion, although they still struggle with contact stability and consistent momentum. Physics strategies are able to fix this, with techniques such as foot locking, trajectory smoothing and contact aware corrections, the simulated motion can be physically plausible and realistic. This is enhanced when integrated into a workflow that also respects the semantic context of the action. This reinforces the need for hybrid methods, which can address both physical and semantic stability after a user edit.

3.3 Limitations in Editing Tools

Current industry tools provide text-prompted generation and simple keyframe editing but do not provide much control over the underlying motion semantics. Unity Muse Animate[23] is a prime example, it can generate motion and then allows pose refinement, although its editing pipeline operates on animation curves after the sequence has been produced. This means the system has no mechanism to reinterpret an edited pose at the generative level, so deeper motion cues such as contact timing, gait consistency and momentum are distorted.

A workflow which respects semantics during editing requires interaction with the generative process itself. In the proposed system, a text-to-motion model such as T2M-GPT[28] is used only for initial synthesis, while a masked reconstruction model such as MoMask[6] performs the short-range regeneration around edited keyframes. MoMask can incorporate the surrounding unedited frames as conditioning context and reconstruct a local interval that blends back into the original motion. Although MoMask is not text-conditioned, it is still effective for keyframe-driven adjustments because the semantic information needed for reconstruction is already present in the neighbouring motion. This stands in contrast

to current industry tools, which lack mechanisms for integrating user edits into generative context.

3.4 Summary and Project Direction

The issues identified in controllability, physical consistency, and existing tools, all point to the need for a hybrid system. The core approach in this project couples two complementary models:

- T2M-GPT[28] for generating the initial text-conditioned motion
- MoMask[6] for local regeneration in response to keyframe edits

When a user adjusts a keyframe, a short transition surrounding the edit is regenerated by MoMask, before the resulting motion is stabilised using IK and physics-aware corrections. This approach provides fine editing capabilities whilst remaining performant enough for interactive use within Unity.

A more ambitious extension would be designing a bidirectional masked transformer that is also text-conditioned, capable of regenerating an internal segment using both preceding and succeeding frames as context. This would enable full segment-level inpainting from text but is recognised as a significant research challenge and is therefore treated as a stretch goal rather than a requirement for the core system.

4 Proposed Solution and Technical Approach

4.1 System Overview

The system developed in this project is an interactive motion editing tool that enables semantic-preservation and physically stable editing inside Unity. This solves the core problem that current industry tools can generate motion, but cannot edit it locally without losing semantics, due to only applying curve-level fixes. The architecture of the core solution will rest on top of two pre-existing models: T2M-GPT[28], which generates the initial sequence, and MoMask[6], which handles the keyframe-guided regeneration of short segments. The reason MoMask has been chosen is because it supports bidirectional context, can easily regenerate transition aligning motion whilst respecting keyframes, and can preserve semantics from the surrounding motion. The conjunction of these two models is paramount, as each cannot do the others job, so a symbiotic relationship is quite efficient.

4.2 Intended Editing Workflow

The full editing pipeline is as follows:

1. User enters a text prompt.
2. T2M-GPT[28] generates a complete motion sequence.

3. The motion is imported into Unity.
4. IK and Physics Corrections are applied.
5. The user designates one of more frames as editable keyframes and modifies their poses.
6. The user presses the "Regenerate Around Keyframes" button.
7. The Unity plugin masks a span of frames around each edited keyframe (with user-configurable padding).
8. The edited keyframe poses are converted into motion tokens via the VQ-VAE codebook and interjected into the masked spans..
9. MoMask[6] regenerates the asked regions using one of the two following methods:
   - Single-pass bidirectional masking: one MoMask[6] pass regenerates both sides of the keyframe span
   - Two-pass regeneration: MoMask[6] is run twice, once to generate the left span (using the keyframe as right anchor), and once for the right span (using the keyframe as left anchor). This is more flexible but may require additional stitching.
10. The regenerated motion is decoded and re-imported into Unity.
11. IK and Physics Corrections are applied.
12. Updated motion is shown in the editor.

This approach allows for precise user edits that preserve semantic intent and remain physically consistent.

4.3 Motion Regeneration Module

MoMask[6] operates by masking spans of tokens in a motion sequence and predicting their values from the surrounding context. It is trained to preserve semantic and temporal consistency while filling in these missing intervals.

In this project, MoMask[6] is used to reconstruct short sequences around user-defined keyframes:

- First, keyframes are quantised into motion tokens.
- The surrounding span (user-defined padding) is masked.
- These masked sequences, with edited keyframes injected in place, are passed to MoMask[6].

I have considered two regeneration options:

Single-Pass Regeneration

- The entire masked region is regenerated in one pass.

- The keyframe tokens are embedded within the masked span and held fixed during generation.
- This allows MoMask[6] to leverage full bidirectional context and jointly optimise both transitions leading into and out of the keyframe.
- Advantages: smooth, continuous transitions on both sides.
- Limitations: may be less predictable if constraints on either side are highly asymmetric, harder to implement as it requires some inference conditioning.

Two-Pass Regeneration

- The masked region is split into left and right halves relative to the keyframe.
- First, the left side is regenerated using the preceding frames as context and anchoring the keyframe at the end.
- Then, the right side is regenerated using the keyframe as the starting anchor and attending to the following frames.
- Each half is reconstructed independently, with the keyframe acting as a semantic hinge.
- There may need to be a narrow overlap region around the keyframe to mitigate any risks of seam discontinuity.
- Advantages: easier to implement as no changes need to be made to the MoMask[6] model.
- Limitation: uses MoMask[6] twice per keyframe, may introduce discontinuity at the keyframe boundary if not smoothed correctly.

While MoMask[6] is not capable of regenerating motion from natural language prompts, it excels at short-range, keyframe-aware regeneration, and can still preserve semantic integrity by taking surrounding motion as context during regeneration.

4.4 Physics and Inverse Kinematics Correction Layer

After generation, motion is stabilised using post-processing techniques including:

- Inverse Kinematics: Ensures joint feasibility and target reach.
- Foot Locking: Enforces ground contact appropriately.
- Trajectory Smoothing: Refines root motion to eliminate jitter or disjointed transitions.
- Velocity Smoothing: Maintains consistent momentum across regenerated spans.

These corrections are applied to any generated motion, not animation curves, meaning the semantic structure remains intact while physical believability is improved. This is a key distinction from tools like Muse Animate, which perform curve-level editing without model-aware regeneration.

4.5 Tools, Libraries and Implementation

- T2M-GPT[28]: Used for initial generation.
- MoMask[6]: Used for bidirectional keyframe segment regeneration.
- Unity (v6+): Editor environment and runtime integration.
- Unity Animation Rigging Package: IK and physics corrections.
- Custom Unity Editor Tools: For keyframe selection and regeneration control.
- PyTorch: Inference and potential fine-tuning.
- Python-Unity bridge: Socket or gRPC communication layer.

4.6 Stretch Goal: Text-Prompted Segment Regeneration

While MoMask[6] can support keyframe-based editing, it does not accept textual prompts. Regenerating a segment from a new instruction (e.g., "do a forward roll here") would require a bidirectional, text-conditioned model.

Two potential approaches include:

- Modifying the T2M-GPT[28]-style pipeline into a bidirectional span-masked architecture, where randomly masked spans are reconstructed using both preceding and succeeding tokens. Text is embedded using a language encoder and injected into the transformer layers to guide regeneration. This mirrors approaches used in masked language models like BERT and MoMask[6], but extends them with dual input streams: text and motion.
- Alternatively, a custom model could be trained directly on masked motion sequences paired with textual prompts. This would require modifying datasets like HumanML3D[5] to simulate editing scenarios. A VQ-based representation (like in MoMask[6]) would likely still be used to tokenise motion.

This type of model introduces much higher complexity, including the need to design or modify bidirectional transformers and cross-modal attention mechanisms that align language with motion features. It may also involve new loss objectives that explicitly enforce semantic consistency between the regenerated motion and the prompt. For this reason it is treated as a stretch goal rather than a core deliverable, though it does represent a meaningful research and industry contribution.

5 Progress and Evaluation Plan

5.1 Achievements to Date

This project has developed a clear and technically grounded approach to build an editing tool for human motion generation. Through an in-depth literature review, key limitations in current models and tools were identified, including the lack of support for fine editing, poor

preservation of semantics after editing, and weak physical realism in existing pipelines. A hybrid architecture was proposed to address these gaps, combining T2M-GPT[28] for initial generation with MoMask[6] for short regeneration span around set keyframes.

A complete editing pipeline has been designed, integrating these models into a Unity extension. The system allows users to generate motion using a prompt, modify poses directly in the editor, and regenerate only the affected segments with preserved context. Techniques for quantising keyframes into motion tokens, flexible masking spans, and different MoMask[6] generation techniques were explored to enhance controllability. The pipeline also includes a second stage for IK and physics correction, ensuring joint feasibility and contact realism. This architecture has been scoped, documented, and technically justified as a feasible and novel solution.

Alongside the core system, a stretch goal has been considered that encompasses the development of a bidirectional, text-conditioned transformer capable of regenerating motion segments based on new prompted instructions. This approach introduces additional architectural complexity, including cross-modal token fusion and span-masked training.

5.2 Evaluation Plan

To rigorously assess the performance and usability of the proposed editing system, the project will adopt two evaluation strategies: quantitative metrics for objective performance, and qualitative user evaluation to assess the editing capabilities of the tool.

5.2.1 Quantitative Evaluation

The following metrics will be used to evaluate the generated or regenerated motion segments:

- Average Displacement Error (ADE) and Final Displacement Error (FDE): Measure the deviation of regenerated segments from ground truth motion, validating whether the generated motions actually align with the prompt inputs.
- Frechet Gesture Distance (FGD): Assesses statistical similarity between regenerated motions and real motions, ensuring semantic realism is preserved after editing.
- Diversity Score: Measures variability across multiple generations with the same prompt to confirm the system avoids output collapse and maintains creative variation.

Through controlled experiments these systems will support systematic benchmarking against existing models and previous work.

5.2.2 Embedded Qualitative Evaluation

A prototype version of the Unity extension will include a lightweight feedback interface, enabling testers to systematically rate and comment on motions generated through various editing operations in a controlled user study. This evaluation tool will collect user input on criteria such as:

- Naturalness and realism of generated and regenerated motions.
- Physical plausibility of the motions.
- Success of preserving semantic intent after edits.
- Perceived control and usability of the editing workflow.

With appropriate ethical clearance, the tool will be distributed to a small number of testers (both experts in ranging appropriate fields and non-experts). Users will be presented with varied scenarios (e.g. walking to running transition, insertion of a new action mid-sequence) and asked to edit and regenerate motion using the tool.

Their responses will be recorded, collated, and visualised to represent the tool's usability and document user experience. This also sets the stage for a qualitative comparison against Unity Muse Animate[23], if time allows.

5.3 Timeline and Remaining Tasks

Weeks	Dates	Milestones and Tasks
11–12	9–20 Dec 2025	• Integrate T2M-GPT[28] and Unity tooling for initial generation • Begin building Unity plugin interface for importing/editing motion • Add initial IK and foot contact stabilisation pipeline
13–14	23 Dec 2025 – 3 Jan 2026	• Finalise keyframe editing tools in Unity • Integrate MoMask[6] for keyframe regeneration • Implement masking pipeline (single vs two pass regeneration)
15–16	6–17 Jan 2026	• Complete regeneration workflow and physics post-processing • Internal testing and debugging • Deliver and document core system for evaluation and write-up
17–18	20–31 Jan 2026	• Develop embedded evaluation toolkit • Conduct structured internal user study • Collect metrics and feedback for both quantitative and qualitative analysis
19–20	3–14 Feb 2026	• Begin prototyping stretch goal: text-driven segment regeneration • Explore bidirectional masked transformer with text prompt integration • Compare results with MoMask[6] baseline
21–22	17–28 Feb 2026	• Refine stretch goal prototype if promising • Integrate into the Unity plugin • Begin writing final chapters of dissertation
23–24	2–13 Mar 2026	• Continue dissertation write-up • Polish evaluation results and visuals • Verify code/documentation for submission

Table 1: Project Timeline (Part 1)

Weeks	**Dates**	**Milestones and Tasks**
25–26	16–27 Mar 2026	• Internal feedback rounds from supervisor and testers • Revise final document
27–28	30 Mar – 10 Apr 2026	• Prepare presentation slides and narration • Continue minor improvements and optional bug-fixes
29–30	13–24 Apr 2026	• Finalise presentation materials • Final backup and archival of project assets • Final supervisor feedback
31–32	27 Apr – 8 May 2026	• Project Presentation: prepare and deliver (6 May)
33	9–13 May 2026	• Final proofreading and submission (13 May)

Table 2: Project Timeline (Part 2)

5.4 Risks and Mitigation

Risk	Impact	Mitigation Strategy
Model Compatibility Issues between MoMask[6] and the Unity pipeline (e.g., tokenisation mismatches or format incompatibilities)	High	Preprocess motions through a standardised latent space encoder. Build bridging tools early and validate with test cases to ensure MoMask[6] integration is robust.
MoMask[6] does not support hard keyframe constraints well, reducing the accuracy of regeneration around edited poses	Medium–High	Implement single or two pass regeneration solutions. If results remain unstable, consider reducing scope to interpolation-only regeneration.
Text-driven segment regeneration stretch goal proves infeasible within time or resource limits	Medium	This goal is explicitly marked as optional. If early tests indicate poor viability, results can still be reported as findings, without jeopardising the core submission.
User study and evaluation data collection delays, especially over term breaks or due to ethical review delays	Medium	Design the embedded evaluation toolkit early. Prepare a small-scale local test plan in case user recruitment is blocked or delayed.
Unity animation tooling limitations, especially in physics corrections and plugin responsiveness	Low–Medium	Build modular Unity components to allow swapping different correction methods. Validate IK and physics correction early in the pipeline.
Overrun on integration or testing, leaving limited time for documentation and polishing	Medium	Set soft deadlines for each phase and keep weeks 25–33 for feedback, refinement and dissertation preparation. Write-up sections iteratively, alongside development.

Table 3: Risks and Mitigation Strategies

6 Conclusion

This project set out to bridge a critical gap in the current landscape of 3D human motion generation, enabling editable, physically plausible, and semantically guided motion directly within a professional animation pipeline. While recent advances in text-to-motion generation have unlocked new creative possibilities, they remain largely static: once generated, changes are driven by legacy techniques. By combining the strengths of autoregressive, text-prompted models like T2M-GPT[28] with MoMask[6]'s bidirectional span-masking capabilities, this system proposes a novel hybrid architecture that enables targeted regeneration around

keyframes, preserving semantic integrity, a workflow far more aligned with how real artists and animators iterate.

The core deliverable provides a Unity-integrated pipeline where users can generate motion from text, edit poses with keyframes, and regenerate adjustable spans around those keyframes with high semantic stability. This is further stabilised through an inverse kinematics and physics correction layer, ensuring that output not only aligns with the user's intentions, but also looks physically believable. The system is designed not as a black-box generator, but a cooperative tool grounded in control and refinement.

Throughout this report, the technical design has been justified by the limitations in current models and industry tools, including an evaluation, a timeline and careful consideration of risks. The ambitious stretch goal of regenerating internal segments using new prompts has been explored through research into a possible bidirection, text-conditioned masked transformer. Even if not fully achieved within the project timeframe, this investigation marks a concrete step toward a more editable and expressive generative system.

To conclude, this project contributes both a practical tool for animators and a framework for editable generative motion. It proposes a unique direction not just in what models can generate, but in how they can be controlled, corrected, and creatively used.

References

[1] Andreas Aristidou and Joan Lasenby. "FABRIK: A fast, iterative solver for the Inverse Kinematics problem". In: *Graph. Model.* 73.5 (2011), pp. 243–260. DOI: 10.1016/J.GMOD.2011.05.003. URL: https://doi.org/10.1016/j.gmod.2011.05.003.

[2] Kevin Bergamin et al. "DReCon: data-driven responsive control of physics-based characters". In: vol. 38. 6. 2019, 206:1–206:11. DOI: 10.1145/3355089.3356536. URL: https://doi.org/10.1145/3355089.3356536.

[3] Stefano Chiaverini. *Estimate of the two smallest singular values of the Jacobian Matrix: Application to damped least-squares inverse kinematics.* Tech. rep. 8. 1993, pp. 991–1008. DOI: 10.1002/ROB.4620100802. URL: https://doi.org/10.1002/rob.4620100802.

[4] Michael Gleicher. "Motion path editing". In: *Proceedings of the 2001 Symposium on Interactive 3D Graphics, SI3D 2001, Chapel Hill, NC, USA, March 26-29, 2001.* Ed. by John F. Hughes and Carlo H. Séquin. ACM, 2001, pp. 195–202. DOI: 10.1145/364338.364400. URL: https://doi.org/10.1145/364338.364400.

[5] Chuan Guo et al. "Generating Diverse and Natural 3D Human Motions From Text". In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR).* June 2022, pp. 5152–5161.

[6] Chuan Guo et al. "MoMask: Generative Masked Modeling of 3D Human Motions". In: *IEEE/CVF Conference on Computer Vision and Pattern Recognition, CVPR 2024, Seattle, WA, USA, June 16-22, 2024.* IEEE, 2024, pp. 1900–1910. DOI: 10.1109/CVPR52733.2024.00186. URL: https://doi.org/10.1109/CVPR52733.2024.00186.

[7] Félix G. Harvey et al. "Robust Motion In-betweening". In: vol. abs/2102.04942. 2021. arXiv: 2102.04942. URL: https://arxiv.org/abs/2102.04942.

[8] Daniel Holden, Taku Komura, and Jun Saito. "Phase-functioned neural networks for character control". In: vol. 36. 4. 2017, 42:1–42:13. DOI: 10.1145/3072959.3073663. URL: https://doi.org/10.1145/3072959.3073663.

[9] Daniel Holden, Jun Saito, and Taku Komura. "A deep learning framework for character motion synthesis and editing". In: vol. 35. 4. 2016, 138:1–138:11. DOI: 10.1145/2897824.2925975. URL: https://doi.org/10.1145/2897824.2925975.

[10] Daniel Holden, Jun Saito, and Taku Komura. "A deep learning framework for character motion synthesis and editing". In: vol. 35. 4. 2016, 138:1–138:11. DOI: 10.1145/2897824.2925975. URL: https://doi.org/10.1145/2897824.2925975.

[11] Biao Jiang et al. "MotionGPT: Human Motion as a Foreign Language". In: (2023). Ed. by Alice Oh et al. URL: http://papers.nips.cc/paper%5C_files/paper/2023/hash/3fbf0c1ea0716c03dea93bb6be78dd6f-Abstract-Conference.html.

[12] Ben Kenwright. "Inverse Kinematics - Cyclic Coordinate Descent (CCD)". In: *J. Graph. Tools* 16.4 (2012), pp. 177–217. DOI: 10.1080/2165347X.2013.823362. URL: https://doi.org/10.1080/2165347X.2013.823362.

[13] Taesoo Kwon and Sung Yong Shin. "Motion modeling for on-line locomotion synthesis". In: (2005). Ed. by Demetri Terzopoulos et al., pp. 29–38. DOI: 10.2312/SCA/SCA05/029-038. URL: https://doi.org/10.2312/SCA/SCA05/029-038.

[14] Zhe Li et al. "OmniMotion: Multimodal Motion Generation with Continuous Masked Autoregression". In: vol. abs/2510.14954. 2025. DOI: 10.48550/ARXIV.2510.14954. arXiv: 2510.14954. URL: https://doi.org/10.48550/arXiv.2510.14954.

[15] Thomas Lucas et al. "PoseGPT: Quantization-Based 3D Human Motion Generation and Forecasting". In: Lecture Notes in Computer Science 13666 (2022). Ed. by Shai Avidan et al., pp. 417–435. DOI: 10.1007/978-3-031-20068-7_24. URL: https://doi.org/10.1007/978-3-031-20068-7%5C_24.

[16] Sihan Ma et al. "Contact-aware Human Motion Generation from Textual Descriptions". In: *CoRR* abs/2403.15709 (2024). DOI: 10.48550/ARXIV.2403.15709. arXiv: 2403.15709. URL: https://doi.org/10.48550/arXiv.2403.15709.

[17] Mathis Petrovich, Michael J. Black, and Gül Varol. "TEMOS: Generating Diverse Human Motions from Textual Descriptions". In: Lecture Notes in Computer Science 13682 (2022). Ed. by Shai Avidan et al., pp. 480–497. DOI: 10.1007/978-3-031-20047-2_28. URL: https://doi.org/10.1007/978-3-031-20047-2%5C_28.

[18] Daniel Rakita, Bilge Mutlu, and Michael Gleicher. "A Motion Retargeting Method for Effective Mimicry-based Teleoperation of Robot Arms". In: (2017). Ed. by Bilge Mutlu et al., pp. 361–370. DOI: 10.1145/2909824.3020254. URL: https://doi.org/10.1145/2909824.3020254.

[19] Sebastian Starke et al. "Neural state machine for character-scene interactions". In: vol. 38. 6. 2019, 209:1–209:14. DOI: 10.1145/3355089.3356505. URL: https://doi.org/10.1145/3355089.3356505.

[20] Guy Tevet et al. *Human Motion Diffusion Model*. 2022. arXiv: 2209.14916 [cs.CV]. URL: https://arxiv.org/abs/2209.14916.

[21] Guy Tevet et al. "Human Motion Diffusion Model". In: *The Eleventh International Conference on Learning Representations, ICLR 2023, Kigali, Rwanda, May 1-5, 2023*. OpenReview.net, 2023. URL: https://openreview.net/forum?id=SJ1kSyO2jwu.

[22] Guy Tevet et al. "MotionCLIP: Exposing Human Motion Generation to CLIP Space". In: *Computer Vision - ECCV 2022 - 17th European Conference, Tel Aviv, Israel, October 23-27, 2022, Proceedings, Part XXII*. Ed. by Shai Avidan et al. Vol. 13682. Lecture Notes in Computer Science. Springer, 2022, pp. 358–374. DOI: 10.1007/978-3-031-20047-2_21. URL: https://doi.org/10.1007/978-3-031-20047-2%5C_21.

[23] Unity Technologies. *Unity Muse Animate: Official Documentation*. https://docs.unity3d.com/Packages/com.unity.muse.animate. 2024.

[24] Jingbo Wang et al. "Scene-Aware Generative Network for Human Motion Synthesis". In: *IEEE Conference on Computer Vision and Pattern Recognition, CVPR 2021, virtual, June 19-25, 2021*. Computer Vision Foundation / IEEE, 2021, pp. 12206–12215. DOI: 10.1109/CVPR46437.2021.01203. URL: https://openaccess.thecvf.com/content/CVPR2021/html/Wang%5C_Scene-Aware%5C_Generative%5C_Network%5C_for%5C_Human%5C_Motion%5C_Synthesis%5C_CVPR%5C_2021%5C_paper.html.

[25] Katsu Yamane. "Simulating and Generating Motions of Human Figures". In: Jan. 2004. ISBN: 978-3-540-20317-9.

[26] Ye Yuan et al. "PhysDiff: Physics-Guided Human Motion Diffusion Model". In: *IEEE/CVF International Conference on Computer Vision, ICCV 2023, Paris, France, October 1-6, 2023*. IEEE, 2023, pp. 15964–15975. DOI: 10.1109/ICCV51070.2023.01467. URL: https://doi.org/10.1109/ICCV51070.2023.01467.

[27] He Zhang et al. "Mode-adaptive neural networks for quadruped motion control". In: vol. 37. 4. 2018, p. 145. DOI: 10.1145/3197517.3201366. URL: https://doi.org/10.1145/3197517.3201366.

[28] Jianrong Zhang et al. "T2M-GPT: Generating Human Motion from Textual Descriptions with Discrete Representations". In: *CoRR* abs/2301.06052 (2023). DOI: 10.48550/ARXIV.2301.06052. arXiv: 2301.06052. URL: https://doi.org/10.48550/arXiv.2301.06052.

[29] Ke Zhang et al. *Think Before You Diffuse: Infusing Physical Rules into Video Diffusion*. 2025. arXiv: 2505.21653 [cs.CV]. URL: https://arxiv.org/abs/2505.21653.

[30] Mingyuan Zhang et al. "MotionDiffuse: Text-Driven Human Motion Generation With Diffusion Model". In: *IEEE Trans. Pattern Anal. Mach. Intell.* 46.6 (2024), pp. 4115–4128. DOI: 10.1109/TPAMI.2024.3355414. URL: https://doi.org/10.1109/TPAMI.2024.3355414.